PANDA: Adaptive Prefetching and Decentralized Scheduling for Dataflow Architectures

SHANTIAN QIN, ZHIHUA FAN, WENMING LI, ZHEN WANG, XUEJUN AN, XIAOCHUN YE, and DONGRUI FAN, State Key Laboratory of Processors, Institute of Computing Technology Chinese Academy of Sciences, Beijing, China and School of Computer Science and Technology, University of Chinese Academy of Sciences, Beijing, China

Dataflow architectures are considered promising architecture, offering a commendable balance of performance, efficiency, and flexibility. Abundant prior works have been proposed to improve the performance of dataflow architectures. Nevertheless, these solutions can be further improved due to the lack of efficient data prefetching and flexible task scheduling. In this article, we propose a novel dataflow architecture with adaptive prefetching and decentralized scheduling (PANDA). First, we present an application-adaptive data prefetching method and on-chip memory microarchitecture designed to overlap memory access latency. Second, we introduce a decentralized dataflow scheduling approach and processing element (PE) microarchitecture aimed at improving hardware utilization. Experimental results show that in a wide range of real-world applications, PANDA attains up to 2.53× performance improvement and 1.79× energy efficiency improvement over the state-of-the-art dataflow architectures.

CCS Concepts: • **Computer systems organization → Data flow architectures**; **Multicore architectures;**

Additional Key Words and Phrases: Prefetching, decentralized dynamic scheduling, reconfigurable on-chip memory architecture

ACM Reference Format:
Shantian Qin, Zhihua Fan, Wenming li, Zhen Wang, Xuejun An, Xiaochun Ye, and Dongrui Fan. 2025. PANDA: Adaptive Prefetching and Decentralized Scheduling for Dataflow Architectures. *ACM Trans. Arch. Code Optim.* 22, 2, Article 62 (June 2025), 27 pages. https://doi.org/10.1145/3721288

1 Introduction

Advances in integrated circuit technology have propelled progress in conventional processors over the past decades. However, this approach is losing effectiveness with the slowdown of Moore's Law [35] and the termination of Dennard Scaling [15]. Fortunately, dataflow architectures exhibit great promise owing to their inherent flexibility and immense potential for high computational parallelism [12, 14].

This paper is sponsored by National Key R&D Program of China (Grant No.2023YFB4503500), Beijing Nova Program (No.20220484054, No.20230484420), Beijing Natural Science Foundation and Changping Innovation Joint Fund (L234078), CAS Project for Youth Innovation Promotion Association.
Authors' Contact Information: Shantian Qin, Zhihua Fan (Corresponding author), Wenming Li (Corresponding author), Zhen Wang, Xuejun An, Xiaochun Ye, and Dongrui Fan, State Key Laboratory of Processors, Institute of Computing Technology Chinese Academy of Sciences, Beijing, China and School of Computer Science and Technology and University of Chinese Academy of Sciences, Beijing, China; e-mails: qinshantian23s@ict.ac.cn, fanzhihua@ict.ac.cn, liwenming@ict.ac.cn, wangzhen21s@ict.ac.cn, axj@ict.ac.cn, yexiaochun@ict.ac.cn, fandr@ict.ac.cn.

ACM 1544-3973/2025/06-ART62
https://doi.org/10.1145/3721288

Dataflow architectures represent a category of architectures that harness the immense potential of high computational parallelism through direct intercommunication among an array of **processing elements (PEs)** [17, 19]. A dataflow program is delineated by a **dataflow graph (DFG)**, which is a graph that depicts operations as nodes and data dependencies as edges. The fundamental principle of the dataflow execution model is that any DFG node can be executed as soon as all its required operands are available [16]. The dataflow architecture primarily comprises a PE array, an on-chip network, an instruction buffer, a data buffer, and a centralized controller. The PE array is composed of numerous PEs interconnected by the on-chip network. It typically operates as an accelerator alongside a host core, collaboratively executing programs. The instruction buffer stores instructions and configuration information, while the data buffer stores the data. The centralized controller manages the scheduling of configuration information, instructions, and data. It also serves as the interface for interactions between the PE array and the host core, facilitating application iteration, initialization, and teardown.

Abundant prior works have been proposed to improve the performance of dataflow architectures. Some works focus on overlapping the memory access latency, employing techniques like ping-pong data buffering [8, 33, 48, 60] to enhance throughput by decoupling and overlapping memory access and execution. Others aim to improve the computational resource utilization, including adopting dataflow-driven scheduling [7, 13, 21, 22, 37, 39] to extract parallelism through dynamic task issuing on hardware. Nevertheless, these solutions can be further improved due to: *(1) Lack of Efficient Data Prefetching*. From the memory access perspective, they commonly rely on **scratchpad memory (SPM)** as on-chip memory, prefetching all data before program execution to overlap memory access latency. However, this approach is constrained by inefficient prefetching of certain data and the finite capacity of on-chip memory. *(2) Lack of Flexible Task Scheduling*. From the computation perspective, they typically adopt pure static scheduling or leverage software and hardware co-design to achieve dataflow-driven scheduling. However, pure static scheduling suffers from insufficient parallelism due to over-serialization and workload imbalance. Moreover, while dataflow-driven scheduling offers greater flexibility than pure static scheduling, it remains hindered by centralized control, resulting in inefficient communication and inflexible hardware.

By conducting a comprehensive review of existing research on data prefetching and task scheduling for dataflow architectures, we aim to explore novel insights and tackle the aforementioned challenges by proposing an innovative dataflow architecture that leverages these insights.

- We find that some data exhibit optimal suitability for prefetching before execution (prefetchable). Conversely, some data are best accessed via cache during execution (non-prefetchable). Driven by this insight, we introduce an *Application-adaptive Data Prefetching Method*.
- We observe that each PE in dataflow architectures possesses the capability to independently schedule tasks, thereby reducing the requirement for a centralized controller. Inspired by this insight, we propose a *Decentralized Dataflow-driven Dynamic Scheduling Method*.

Building upon the previously discussed insights, we propose a novel dataflow architecture with adaptive **p**refetching **an**d **d**ecentr**a**lized scheduling (**PANDA**) as a solution that aptly aligns with our goal.

Our contributions are as follows:

- We introduce an application-adaptive data prefetching method: (1) It describes memory access interfaces, supporting the prefetch/postback and load/store for prefetchable and non-prefetchable data. It fosters the decoupling of the prefetchable data access from the application execution. (2) It constructs an on-chip memory microarchitecture tailored for multi-core accelerators based on the ISA. It leverages reconfigurable physical storage shared by SPM and cache to reduce on-chip area overhead. (3) It presents a mechanism for

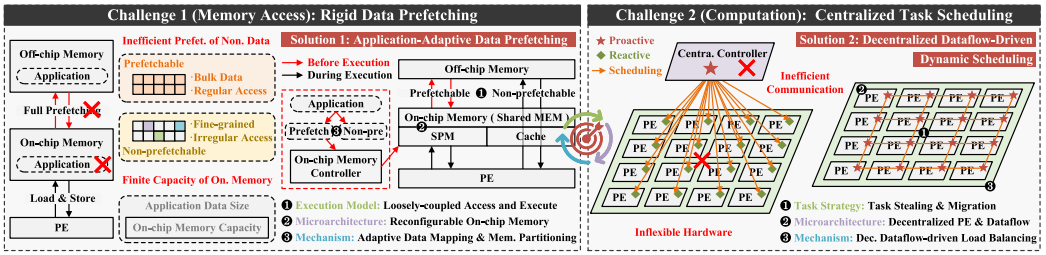

Fig. 1. Traditional dataflow architecture, challenges, and solutions into PANDA design.

application-adaptive data prefetching and on-chip memory partitioning. It facilitates the organized separation of data into prefetchable and non-prefetchable categories.

— We propose a decentralized dataflow-driven dynamic scheduling method: (1) It describes task stealing and migration strategies based on PE state. It facilitates better workload balancing through on-demand real-time task scheduling for busy and idle PEs. (2) It constructs a decentralized PE microarchitecture. It enhances hardware flexibility by empowering PEs to autonomously and proactively schedule their internal tasks. (3) It presents a mechanism for decentralized load balancing. It effectively integrates the aforementioned strategies with hardware microarchitecture.

— We implement all modules of PANDA using Verilog. Moreover, we conduct a comprehensive evaluation that includes the acceleration effect of each innovation feature. Compared to the state-of-the-art dataflow architectures, PANDA outperforms REVEL, Plasticine, DFU, and MTDE by geomean 2.53×, 1.90×, 1.38×, and 1.19× in a wide range of real-world applications, including scientific computing, artificial intelligence, digital signal processing, and graph processing algorithms.

2 Background and Related Works

In this section, we delve into related works concerning data prefetching and task scheduling for dataflow architectures to identify challenges for performance improvement and explore novel insights. Following this analysis, we introduce our proposed architectural overview, focusing on two primary innovations: application-adaptive data prefetching and decentralized dataflow-driven dynamic scheduling.

2.1 Data Prefetching: Overlap Memory Access Latency

The Memory Wall [57] remains a significant barrier to system performance improvement. This challenge is particularly accentuated in multi-core accelerators due to the increased demand for memory bandwidth and the necessity for data sharing among multiple cores on the same chip [6]. As the number of cores increases, this issue intensifies further, imparting a detrimental impact on the overall performance and efficiency of multi-core systems [11, 45]. Additionally, efficient on-chip memory solutions can help bridge the speed gap between processors and memory by reducing memory access latency and optimizing data transfer. Consequently, designing efficient on-chip memory solutions for multi-core accelerators is essential to mitigating the impact of memory wall and enhancing their overall performance. In the realm of on-chip memory solutions, cache and **scratchpad memory (SPM)** stand out.

Cache has been widely used in modern systems as an effective strategy for mitigating the impact of the memory wall [3, 23, 45, 59], which is *globally addressed*. However, cache has several

weaknesses: *(1) Indirect Addressing:* Cache loads and stores specify addresses that hardware must translate to determine the physical location of the accessed data; *(2) Implicit Data Orchestration:* The load request initiators do not directly govern the cache hierarchy's determinations regarding the retention or removal of response data at different storage hierarchy levels [46].

An alternative to the cache is the SPM [8, 10, 17, 18, 20, 26, 37, 41]. SPM is *directly addressed* so it does not have overhead from tag comparisons, which provides significant savings: 34% area and 40% power[4]. SPM also provides *explicit data orchestration* (data in SPM are explicitly allocated by software) and support for rapid and simple *software prefetching of bulk data* as it is managed in software, either by the programmer or through compiler support [42]. These features make SPM appealing, especially for real-time systems [27]. However, SPM also has some inefficiencies: *(1) No Global Addressing:* SPM uses a separate address space disjoint from the global address space, with no hardware mapping between the two; *(2) No Irregular Access Pattern:* The order of accesses to SPM must be known statically upfront and cannot be conditional. The long setup time of **direct memory access (DMA)** leads to the fact that for *fine-grained* data access, *hardware on-demand load* of cache during execution is more efficient than software prefetching of SPM.

Based on the above analysis, cache and SPM each have distinct strengths and weaknesses, excelling only in specific scenarios and lacking broad applicability across various scenarios. There are two main options for combining the advantages of SPM and cache: integrating cache into the on-chip memory architecture alongside SPM, and designing a new on-chip memory architecture that merges the strengths of both SPM and cache. To fully leverage the benefits of cache and SPM, the on-chip memory architecture that integrates both cache and SPM has been proposed and extensively embraced within the industry [2, 38]. However, this configuration significantly amplifies the on-chip area overhead. In addition, the pursuit of an on-chip memory architecture that combines the merits of SPM and cache has garnered significant research attention. There have numerous prior works focused on enhancing on-chip memory architecture by amalgamating the benefits of cache and SPM. ROMA [44] conducted a comprehensive review and comparison of related works, focusing on *Globally Addressed SPM* (e.g., Stash [31]), *Reconfigurable Cache* (e.g., Jenga [50] and Hotpads [51]), and *Decoupled Access Execute (DAE) Architectures* (e.g., DeSC [25] and PDAE [9]). However, none of them fully encompasses all of the benefits illustrated in Table 1.

Challenge1: Traditional dataflow architectures commonly rely on SPM as on-chip memory, prefetching all data to SPM before program execution to overlap memory access latency. However, this approach encounters several limitations, as illustrated in Figure 1 (left). First, not all data can be effectively prefetched, especially small-scale data scattered in off-chip memory. Second, the finite capacity of on-chip memory imposes constraints, particularly for applications with large data sizes, where prefetching all data is impractical. For kernels like matrix multiplication, large data can be split into smaller chunks that fit within on-chip memory for iterative execution. However, for kernels with irregular memory access, such as **Breadth First Search (BFS)**, data access is dynamic and unpredictable. These workloads often involve numerous random data accesses across diverse memory regions, making it difficult to split the data into smaller chunks for iterative execution.

By analyzing the characteristics of numerous real-world applications, we have identified a specific data storage and memory access pattern that exhibits regularity. This pattern involves data being stored and accessed in the form of distinct blocks, each characterized by uniform block size and an identical stride length separating consecutive blocks. For this type of data, provided that the start address, block size, stride length, and the number of blocks are known, it becomes feasible to compute the addresses of all data elements. In previous studies, this pattern is defined as *stream* [37, 52–55]. By prefetching this type of data into on-chip memory before program execution, we

Table 1. Comparison of Cache, SPM, and PANDA

Features	Cache	SPM	PANDA
Global Addressing	✓	✗	✓ (Non-pre.)
Irregular Access Pattern	✓	✗	✓ (Non-pre.)
Hardware Fine-grained Load	✓	✗	✓ (Non-pre.)
Software Bulk Prefetching	✗	✓	✓ (Prefetch.)
Direct Addressing	✗	✓	✓ (Prefetch.)
Explicit Data Orchestration	✗	✓	✓ (Prefetch.)
Reconfigurable Physical Storage	✗	✗	✓

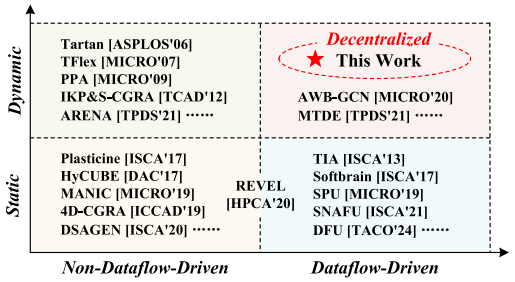

Fig. 2. Scheduling approaches of current research.

can reduce the number of requests and transfers in the memory system, thereby diminishing memory access latency and increasing memory access efficiency.

We observe that the proportion of prefetchable and non-prefetchable data exhibits variability across diverse applications. First, prefetchable data are prevalent and constitute a substantial portion in some applications, such as General Matrix Multiplication, Stencil, Rectified Linear Unit Activation, and Secure Hash Algorithm 256. SPM with DMA supports prefetch before execution to access prefetchable data more efficiently than cache. This highlights the advantage of burst transfer for bulk data. Additionally, non-prefetchable data are also not negligible and account for a large proportion of some applications, particularly in applications like Lagrange Interpolation, BFS, and **PageRank (PR)**. Cache supports load/store during execution to access non-prefetchable data more efficiently than SPM. This is primarily because, in cases of fine-grained data transfers, the burst length is often minimal or even there is no burst transfer. In such scenarios, the total DMA initial time overhead resulting from prefetch becomes very large.

To adeptly accommodate this diversity, we introduce an application-adaptive data prefetching method and design a novel reconfigurable on-chip memory architecture tailored for dataflow architectures, amalgamating the logical organization of SPM and cache while sharing physical storage. The reconfigurable on-chip memory architecture provides support for both prefetching before execution and load/store during execution, ensuring efficient memory access to both prefetchable and non-prefetchable data, as depicted in Figure 4. It is engineered to deliver the benefits of both SPM and cache without incurring excessive area overhead, as shown in Table 1. It is essential to recognize that the data size of prefetchable candidates in certain applications exceeds the storage capacity of on-chip memory. For kernels like matrix multiplication, large data is split into smaller chunks that fit within on-chip memory for iterative execution. However, for kernels like BFS with irregular memory access, splitting the data into smaller chunks for iterative execution is challenging. In such scenarios, proactive endeavors are undertaken to prefetch as many prefetchable candidates as possible from off-chip to on-chip memory before execution. Prefetchable candidates that cannot be prefetched and remain in off-chip memory are then reclassified as non-prefetchable.

In addition, there are key differences that distinguish PANDA prefetching from out-of-order memory accesses to hide memory access latency. In an out-of-order core, memory access latency can be hidden by reordering instructions within the instruction window, but each load instruction still requires tag-checking in caches and frequent fine-grained off-chip memory access. In contrast, adaptive prefetching in PANDA directly links bulk prefetchable data with the Prefetch instruction, fully leveraging the rapid bulk prefetching capabilities of DMA. Furthermore, the on-chip SPM functions as software-exposed stream buffers, eliminating the overhead of dynamic prediction and tag-checking in caches. Additionally, prefetchable data access is decoupled from the execution pipeline. This decoupling is particularly beneficial in mitigating the negative impact of long-latency memory accesses, without requiring a large instruction window. As a result, PANDA

prefetching reduces the instruction pressure on the execution pipeline and enables more efficient memory access.

2.2 Task Scheduling: Improve Computational Utilization

In dataflow architectures, where computational resources are not only abundant but also spatially distributed, improving resource utilization is paramount for enhancing performance and energy efficiency [19]. The choice of task scheduling method plays a pivotal role in this improvement, as it determines the spatial-temporal allocation of workload to hardware architecture [33]. In recent years, numerous studies have focused on enhancing the computational resource utilization of dataflow architectures through optimized task scheduling methods. In this section, we will delve into these works in detail.

Scheduling in dataflow architectures presents greater challenges compared to traditional processors, as it involves both task issuing and task mapping (allocating tasks to spatial resources). Task issuing determines the exploitable parallelism, while task mapping influences resource utilization [7]. In dataflow architectures utilizing pure *static scheduling* [24, 28, 29, 43, 55], the issuing and mapping of tasks are handled statically by the compiler. While these architectures benefit from low hardware overhead, they often suffer from insufficient parallelism due to *over-serialization* and *workload imbalance*. On the one hand, the compiler must create conservative schedules to accommodate the worst-case scenario, leading to over-serialized execution. This is particularly problematic for applications with irregular computation patterns, such as tree and graph algorithms. On the other hand, for large-scale dataflow architectures [5, 47], tasks are typically generated by the compiler and statically mapped to tiles, which are subarrays of interconnected PEs. In such scenarios, achieving workload balance becomes critical for maintaining high throughput. However, achieving balanced execution becomes challenging due to inter-task and intra-task computations variance [40]. Moreover, workloads often change with iteration in irregular applications, rendering pure static scheduling impractical for balanced execution.

For *over-serialization*, some efforts have effectively leveraged the intrinsic characteristics of dataflow architectures by employing a hardware-software co-design approach to achieve *dataflow-driven scheduling* [7, 13, 19, 21, 22, 37, 39]. Instead of relying on the compiler, the dataflow architecture of dataflow-driven scheduling extracts parallelism by dynamic task issuing on hardware. This approach determines when to execute a specific operation based entirely on the control of dataflow, rather than relying on static planning and prediction in advance. It is worth mentioning that REVEL [56] comprises simple PEs in a systolic form (non-dataflow-driven) and dataflow PEs capable of performing complex computations (dataflow-driven), achieving the hybrid static-dynamic task issuing. However, it heavily relies on the compiler's heuristic search for the globally optimal timing decision due to the consideration of balancing between dataflow PEs and systolic PEs, making it significantly less flexible than true dataflow-driven scheduling.

For *workload imbalance*, certain works adjust the allocation of resources and workloads through dynamic mapping during execution to achieve *dynamic scheduling* [1, 7, 21, 30, 34, 40, 49]. There are mainly two kinds of dynamic mapping methods: *(1) Software-driven:* This approach relies on the software compiler. Examples include heuristic virtualization [34], which periodically searches for an optimized mapping to maximize a predefined function, and the bitstream transformation mechanism [40], which allows the shrinking and growing of resources for individual tasks. A common limitation of these approaches is that they lack a detection mechanism to monitor task execution. They statically estimate the workloads of each task, leading to suboptimal mapping when the workload varies and is unpredictable due to dynamic computation variance. Additionally, the algorithmic complexity and sensitivity to function forms and parameters make these approaches less favorable for hardware designs; *(2) Hardware-driven:* This approach is based on real-time hardware

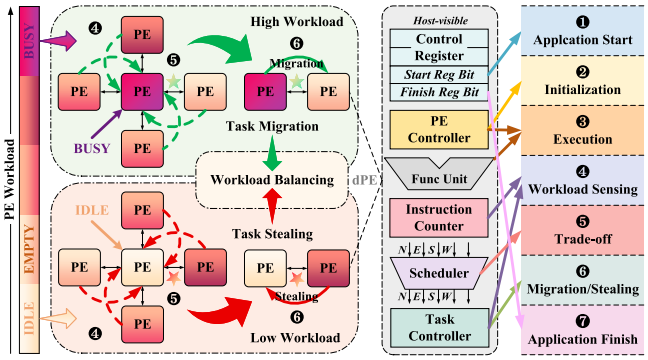

Fig. 3. General workflow of workload-based task migration & stealing and components of decentralized PE.

execution. An example is MTDE [7], which implements an elastic task scheduling scheme. This approach dynamically reschedules tasks at runtime using the tagged-token dataflow paradigm to support dynamic task-level parallelism. Tasks are also dynamically resized during execution through duplication, combination, and substitution operators, ensuring balanced multi-task execution. These hardware-driven methods [7, 21] are more adaptive to applications because decisions are based on real-time workload variations, and the logic is straightforward for hardware designs.

The difference between static and dynamic scheduling lies in whether task mapping is statically predetermined before execution and fixed during execution or can be dynamically adjusted during execution, while the distinction between non-dataflow-driven and dataflow-driven scheduling pertains to whether task issuing is statically pre-planned and compiled in advance or dynamically driven by dataflow during execution. Therefore, existing scheduling schemes in dataflow architectures are classified into four types in Figure 2.

In traditional dataflow architectures, communication between the host core and dataflow PE array, comprised of multiple PEs, is typically facilitated by a centralized controller, including the stream dispatcher & engine in Softbrain [37], the control core in Fifer [36], the stream control unit in REVEL [56], the micro-controller in DFU [19], and the task scheduling & control unit in MTDE [7]. These controllers play several key roles in traditional dataflow architectures. They manage the scheduling of configuration information, instructions, and data through a centralized control mechanism. They also act as the hub for interactions between the PE array and the host core, facilitating application iteration, initialization, and teardown.

Challenge2: While dataflow-driven dynamic scheduling offers greater flexibility than pure static scheduling, it still suffers from centralized control, as shown in Figure 1 (right). First, centralized control necessitates frequent interactions between the centralized controller and PEs, resulting in significant communication and routing overheads. This also leads to increased latency during execution. These issues become increasingly problematic as the scale of PE arrays continues to expand. Second, PEs under centralized control cannot autonomously schedule internal tasks. This absence of autonomous and proactive task scheduling limits hardware flexibility, impeding the achievement of asynchronous inter-PE communication and on-demand real-time scheduling. Achieving workload balancing becomes more arduous in this scenario as well.

We observe that PEs in dataflow architectures are generally sophisticated, and equipped with internal control units and interconnected data paths between PEs. Consequently, each PE possesses the potential to schedule internal tasks independently. However, PEs currently lack the ability to autonomously and proactively schedule tasks, instead relying on reactive scheduling under centralized control.

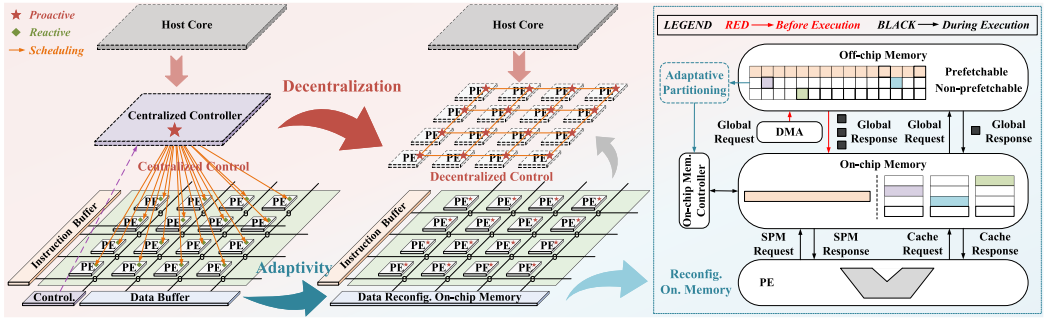

Fig. 4. PANDA overview: Comparison of traditional dataflow architecture and PANDA architecture.

Driven by this insight, we realize that the centralized controller can be replaced by the PE capable of proactive task scheduling to achieve decentralized dataflow-driven dynamic scheduling. To fully harness the advantages of decentralized scheduling, we introduce strategies for dynamic task migration and stealing based on PE workload. To support the decentralized scheduling method, we present a decentralized PE microarchitecture, as illustrated in Figure 3. This decentralized PE enables direct interaction with the host core and autonomous, proactive scheduling of configuration information, instructions, and data.

Through a comprehensive review of existing research on data prefetching and task scheduling for dataflow architectures, we identify two challenges for performance improvement from memory access and computation perspectives. Additionally, we uncover two novel insights to address these challenges and propose an innovative dataflow architecture, PANDA, which leverages these insights, as illustrated in Figure 4. PANDA features two primary innovations: application-adaptive data prefetching and decentralized dataflow-driven dynamic scheduling. We will describe these innovations in detail in the following sections.

3 Adaptive Prefetching

In this section, we first describe an ISA and its programmability, which provides support for prefetch/postback before/after execution and load/store during execution. By extending the memory access ISA, we not only enhance the abstraction level and reduce hardware design complexity but also optimize the execution model by decoupling prefetchable data access from computation.

Subsequently, we construct a reconfigurable on-chip memory microarchitecture with the two primary design principles: We avoid excessive additional area overhead increase, especially for the SRAM, and we have both SPM and Cache corresponding to the two types of memory access commands in the ISA. Here we describe our reconfigurable on-chip memory microarchitecture implementation.

Finally, we present a mechanism for application-adaptive data prefetching and on-chip memory partitioning in applications. It effectively categorizes the data necessitating access during application execution into the two aforementioned categories: prefetchable and non-prefetchable.

3.1 Execution Model of Loosely-coupled Access and Execute

The PANDA ISA abstracts data movement before/after execution and during execution with prefetch/postback and load/store. Table 2 lists the operations provided by PANDA ISA. We decouple prefetchable data access from execution by extending the access ISA with the introduction of Prefetching and Postback instructions. This optimization enhances the execution model and effectively reduces memory access latency.

Table 2. PANDA Instruction Extension

Command Names	Parameters	Description
PANDA_Prefetch	Source Off-chip Memory Address, Step Length, Access Size, Steps, Destination On-chip Memory Address	Data transfer from off-chip memory to on-chip memory before execution
PANDA_Postback	Source On-chip Memory Address, Step Length, Access Size, Steps, Destination Off-chip Memory Address	Data transfer from on-chip memory to off-chip memory after execution
PANDA_Load	Source Address, Destination Register Identifier, Access Selection Bit	Load prefetchable data from on-chip memory or Load non-prefetchable data from off-chip memory
PANDA_Store	Source Register Identifier, Destination Address, Access Selection Bit	Store prefetchable data to on-chip memory or Store non-prefetchable data to off-chip memory

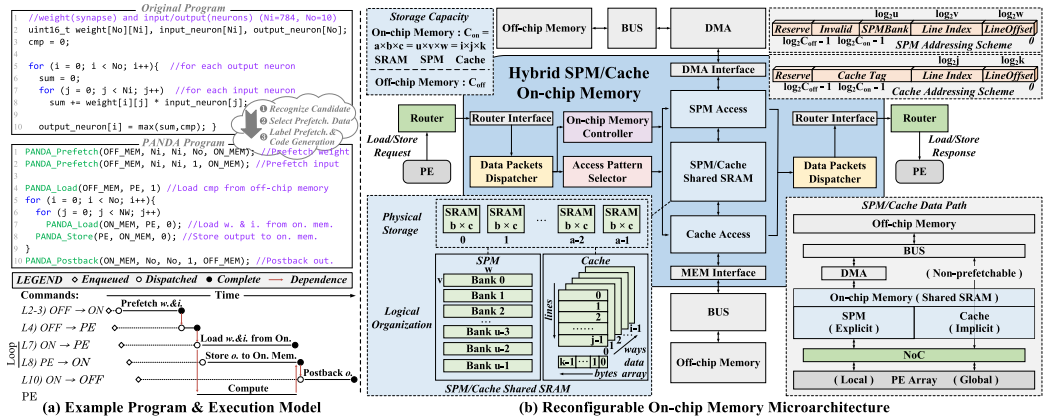

Fig. 5. PANDA memory: (a) Example program & execution model. (b) Reconfigurable microarchitecture.

Prefetch and Postback. Prefetch and *Postback* adopt the two-dimensional affine access, defined by access size (size of lowest level access), step length (size between consecutive accesses), and the number of steps. In addition, the *Prefetch* command specifies the source off-chip memory address and destination on-chip memory address, while the *Postback* command specifies the source on-chip memory address and destination off-chip memory address.

Load and Store. PANDA supports accessing individual elements and loading them into PE registers or storing them into on-chip memory. *Load* or *Store* commands specify a register target and source or destination address.

Example Program. Figure 5(a) illustrates an example neural network classifier, which primarily involves a dense matrix-vector multiplication of synapses (weights) and input neurons. The figure shows the original program, PANDA version, and execution behavior. The transformation to a PANDA version pulls the loading of the weight and input arrays out of the loop using *Prefetch* command (line 2-3) and local *Load* command (line 7). This not only reduces the overhead of requiring long instruction windows in the execution pipeline, but also decreases the number of inefficient and fine-grained data access requests to off-chip memory, thus improving overall memory access efficiency.

In addition, we implement compiler support to identify the prefetchable data and transform the original program to PANDA program. Our implementation uses LLVM [32]. There are three phases: recognizing candidates for prefetchable data, selecting prefetchable data, and code generation & labeling prefetchable data.

Recognizing Candidates for Prefetchable Data. The compiler first analyzes all static memory access instructions within loops, treating each instruction as a candidate for prefetchable data. By

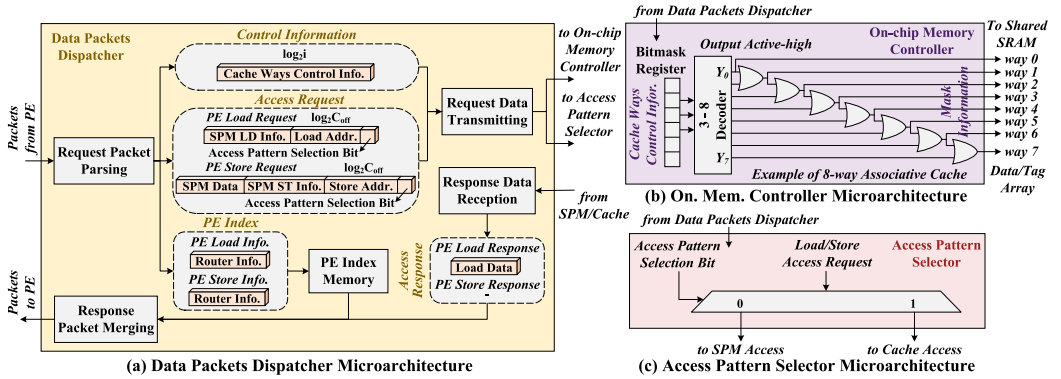

Fig. 6. Dispatcher, controller, and selectore microarchitectures.

evaluating the address, size, and stride of these instructions, it identifies those that conform to the two-dimensional affine access pattern as final prefetchable candidates. Furthermore, the compiler coalesces prefetchable candidates that share the same induction variable and have a small offset between their elements to optimize memory access.

Selecting Prefetchable Data. Due to the limited capacity of on-chip memory, it is necessary to determine whether the total size of the candidates qualified for prefetching exceeds the available on-chip memory capacity. If the total data size exceeds the on-chip memory capacity, an approach that combines memorization search with knapsack dynamic programming techniques will be employed to prefetch as much data as possible. Further details will be described in Section 3.3.

Code Generation & Labeling Prefetchable Data. During this phase, the compiler first generates the prefetching configuration for the selected prefetchable data. The configuration specifies parameters such as address, size, and stride. Then, based on the selected prefetchable data and their associated parameters, the compiler generates *Prefetch* commands and labels these prefetchable data by setting the access pattern selection bit of the *Load* commands to 0. This ensures that during execution, these data can be directly accessed from the on-chip SPM.

3.2 Reconfigurable On-Chip Memory Microarchitecture

At a high level, we combine a data packets dispatcher to assist the interconnection of on-chip memory and **processing element (PE)**, an on-chip memory controller to configure the physical storage, an access pattern selector to allocate the access request to SPM or cache, the SPM with DMA to support prefetch/postback for prefetchable data, the cache to support load/store for non-prefetchable data, and reconfigurable physical storage shared by SPM and cache (see Figure 5).

3.2.1 Data and Packets Dispatch. PEs are typically interconnected through the **network on chip (NoC)** to facilitate information exchange between PEs and on-chip memory through packets. However, the internal units of the reconfigurable on-chip memory communicate primarily through data rather than packets. The data packets dispatcher serves the critical role of converting between data and packets, requiring the parsing and merging of packets from/to PEs, as well as data exchange with the other internal units. The data packets dispatcher fulfills the external interaction of packets with PEs, as well as the internal interaction of data with the controller, selector, SPM, and cache.

The design of the data packets dispatcher is depicted in Figure 6(a). Initially, request packets from PEs are parsed into data. This unit transmits data, including access request information and

control signals, to the selector and controller. It stores PE index information in its internal memory. Furthermore, it receives data associated with access responses from other units. Finally, it merges the stored and received data into response packets destined for PEs.

3.2.2 On-Chip Memory Control & Access Pattern Selection. The on-chip memory controller plays a vital role in managing the SRAM that contains prefetchable data. It incorporates a bitmask register that holds the software-generated on-chip memory configuration information. This bitmask is used to configure specific cache ways as SPM, excluding them from cache lookup and replacement. This mechanism ensures that the prefetchable data accessed through SPM will not be overwritten by non-prefetchable data accessed through the cache.

Request packets from the PE carry SPM/Cache access pattern selection information (the lowest bit of the access request). The data packets dispatcher, upon parsing the access request, conveys the access pattern selection bit to the access pattern selector. If this bit is 0, the access pattern selector sends the load/store access request to the SPM; otherwise, it sends the request to the cache.

The controller flexibly and dynamically allocates the on-chip storage resources based on the prefetchable data size of the application, and the selector routes the access request from the PE to the SPM or cache. They are critical to achieving high generality with low area overhead. By partitioning the on-chip SRAM into SPM and cache based on the prefetchable data size of the application to access the prefetchable and non-prefetchable data, efficient resource utilization is achieved.

3.2.3 SPM and Cache Logical Organization. Our design incorporates both SPM and cache, each with distinct logical organizations, while sharing the same physical storage. SPM contains u banks, each of which is comprised of v lines with w bytes width. SPM interacts with off-chip memory through DMA and with PE array through NoC, where the former can be realized by *Prefetch* and *Postback* commands before/after execution, and the latter can be realized by *Load* and *Store* commands during execution. Cache is organized in i-way set-associative and each way consists of j lines with k bytes width. The interaction between cache and off-chip memory is facilitated by BUS, while its communication with PE array is enabled through NoC. The above two can be realized by *Load* and *Store* commands during execution.

Data Path. Our design provides two distinct data paths, SPM with DMA and cache, to facilitate the access of prefetchable and non-prefetchable data, respectively. Before application execution, DMA in the SPM path efficiently prefetches all prefetchable data from off-chip memory to on-chip memory. Additionally, the SRAMs containing the prefetchable data are configured as SPM and are excluded from cache lookup and replacement by the on-chip memory controller. During execution, PE accesses the prefetchable data directly from on-chip memory through SPM path, while PE accesses the non-prefetchable data from off-chip memory through cache path. This comprehensive setup ensures efficient access to both categories of data.

Coherence amd Consistency. In our approach, the compiler identifies and labels prefetchable data prior to program execution. These labeled data are then prefetched into on-chip memory. The on-chip SRAM storing prefetchable data belongs to the SPM access space, and the SPM capacity is determined by the size of the prefetchable data. The remaining on-chip SRAM is used for cache access and handles non-prefetchable data during execution. To ensure separation between SPM and cache, we utilize a cache way masking mechanism. This mechanism employs a bitmask register that stores software configuration information to control which cache ways are active during the lookup and replacement phases. This allows software to designate certain cache ways as SPM, excluding them from cache lookup and replacement. Consequently, the SPM is explicitly managed by software, while the cache remains implicitly managed by hardware and requires a hardware coherence protocol. PANDA adopts the MESI protocol for cache coherence.

3.2.4 SPM and Cache Physical Storage. The total storage capacity of SPM is $u \times v \times w$ bytes, and the total storage capacity of cache is $i \times j \times k$ bytes. In our design, SPM and cache adopt a unified physical storage, which is called shared SRAM. This shared SRAM consists of a individual SRAM units, each with a depth of b and a width of c bytes. Its total storage capacity is $a \times b \times c$ bytes. Note that the total storage capacity of SPM, cache, and shared SRAM must be equal.

Addressing Scheme. Since SPM and cache use the same physical storage, we need to address them separately based on SPM and cache logical organization. SPM addressing primarily involves banks, line index, and offset (u, v, w), while cache addressing mainly focuses on cache line index and offset (j, k). Assuming that off-chip memory storage capacity is C_{off} bytes and on-chip memory storage capacity is C_{on} bytes. Cache supports off-chip memory global addressing and uses $log_2 C_{off}$ bits valid address for addressing. In contrast, SPM supports only on-chip memory local addressing, and its valid addressing bit width is limited to $log_2 C_{on}$. Additionally, we reserve higher bits of the addressing for potential optimizations and specific design features. For instance, if the on-chip memory adopts a double-buffering technique, an extra bit might be needed to distinguish between the two buffers.

3.3 Adaptive Data Mapping & On-chip Memory Partitioning

In this section, we describe a mechanism for adaptive data mapping and on-chip memory partitioning before application execution. It outlines the process of selecting prefetchable data and generating the necessary on-chip memory configuration information for the on-chip memory controller, thus completing the overall workflow.

The initial step involves recognizing candidates for prefetchable data mapped to the SPM path, with the cumulative size of the candidates being calculated. During this process, the compiler evaluates the address, size, and stride of each memory access, recognizing those that conform to the two-dimensional affine access pattern as prefetchable candidates. Simultaneously, the remaining data are classified as non-prefetchable and mapped to the cache path.

To safeguard the integrity of prefetchable data from modification by non-prefetchable data during execution, the cache ways containing prefetchable data are configured as SPM and excluded from cache lookup and replacement. However, since non-prefetchable data must be accessed via the cache, it is essential to ensure that at least one cache way remains active during the cache lookup phase and possesses proactive global addressing capability. In scenarios where the total size of prefetchable data is excessively large, we employ an approach that combines memorization search with knapsack dynamic programming ideas. Initially, the compiler explores all potential scenarios through dynamic programming and subsequently traces back to identify the scenario that maximizes the prefetchable data size. Additionally, it backtracks to label all prefetchable data candidates within this optimal scenario.

Ultimately, by evaluating the size of prefetchable data, the compiler determines the number of cache ways that must be configured as SPM and generates the cache ways control information for the on-chip memory controller.

4 Decentralized Scheduling

In this section, we first introduce dynamic task migration and stealing strategies based on PE state to better achieve workload balancing through on-demand real-time task scheduling for busy and idle PEs.

Subsequently, we construct a decentralized PE microarchitecture with the following goal: empowering PEs to autonomously and proactively schedule their internal tasks to improve hardware

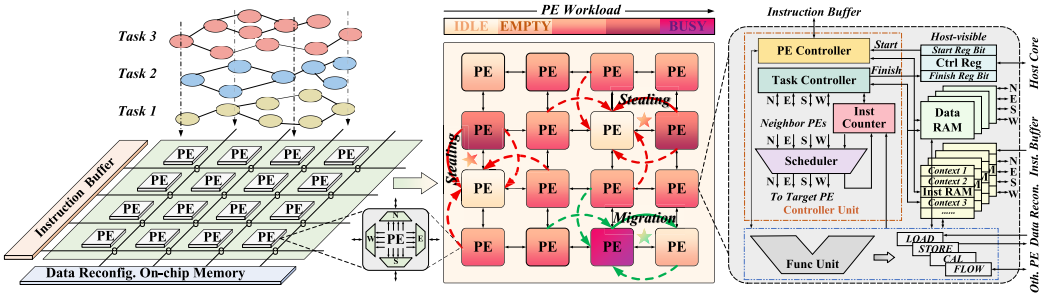

Fig. 7. Workload-based decentralized dataflow-driven task migration/stealing strategies and decentralized PE microarchitecture.

flexibility. Here we describe our decentralized PE microarchitecture implementation and the overall procedure flow based on decentralized PE microarchitecture.

Finally, we propose a decentralized load balancing mechanism. It effectively integrates the aforementioned strategies with hardware microarchitecture in the context of multiple tasks to enhance computational resource utilization.

4.1 Workload-Based Task Stealing & Migration

Task Model. Multiple tasks are assigned to the PE array, with each task being executed collectively by one or more PEs. Each task is represented as a **dataflow graph (DFG)**, as illustrated in Figure 7 (left). Consequently, a single PE may handle segments of various DFGs, and all segments of a DFG assigned to a specific PE are termed as a context. In addition, we use the application-based multi-DFG program execution model [19], where an application consists of multiple sequentially executing tasks. Each task is a DFG comprising multiple computational nodes and directed edges, and it may be executed iteratively multiple times.

First, the various workload states of the PE are delineated, as shown in Figure 7 (middle). Specifically, we define three key states: *IDLE*, *EMPTY*, and *BUSY*. A PE transitions to the *IDLE* state once it has executed all its assigned contexts. Similarly, a PE transitions to the *EMPTY* state upon executing the final context, signifying that there are no remaining contexts. Conversely, a PE enters the *BUSY* state when the number of remaining contexts surpasses the busy threshold. This threshold is dynamically adjusted based on the number of *EMPTY* and *IDLE* PEs present on the current array.

Second, novel workload balancing strategies tailored for the decentralized dataflow architecture are introduced. The objective is to optimize workload balancing by ensuring that more PEs are in a balanced state rather than being in a high-workload (*BUSY*) or low-workload (*IDLE*) state. To achieve this, two workload balancing strategies are proposed: task migration for *BUSY* PEs and task stealing for *IDLE* PEs. It facilitates better workload balancing through on-demand real-time task scheduling for *BUSY* and *IDLE* PEs.

To better support decentralized dataflow-driven task stealing and migration, a decentralized PE microarchitecture is constructed, as illustrated in Figure 7 (right). Detailed elaboration on this microarchitecture will follow in the next section.

4.2 Decentralized PE Microarchitecture

We divide the execution process of a multi-DFG application into seven stages: Application Start, Initialization, Execution, Workload Sensing, Trade-off, Stealing/Migration, and Application Finish. To facilitate the execution of these stages, we design the decentralized PE microarchitecture. This

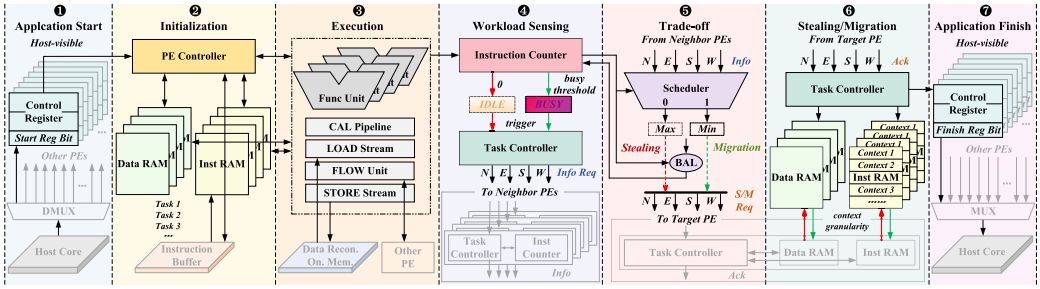

Fig. 8. Overall procedure flow (seven stages) based on decentralized PE microarchitecture.

microarchitecture is primarily composed of three modules: Controller Unit, Memory Unit, and Function Unit. Next, we provide a detailed introduction to each of these modules. Lastly, we elucidate the overall procedure flow (seven stages) based on decentralized PE, as depicted in Figure 8.

Controller Unit. This unit comprises four components, including a PE controller to initialize configuration information and instructions, a task controller to manage workload sensing and stealing/migration, an instruction counter to record the remaining contexts and instructions of the PE, and a scheduler to make workload trade-off decisions.

Memory Unit. This unit encompasses a host-visible control register for the PE to interact with the host core, a data RAM to store data, and an instruction RAM to store instructions.

Function Unit. This unit includes a calculation pipeline to generate the data path for arithmetic operations and logical operations, a load/store stream to transfer data from/to the data reconfigurable on-chip memory, and a flow unit to dispatch data to downstream PE, decoupling the execution of a DFG node into up to four consecutive phases: Load, Calculating, Flow and Store.

Overall Procedure Flow (Seven Stages):

❶ *Application Start.* The host core triggers the start of the application by setting the start register bit of the control register of each PE to 1, indicating the start of execution. This signal prompts the PEs to begin their respective tasks.

❷ *Initialization.* Upon receiving the start signal from the host core, each PE retrieves necessary instructions and configuration information from the instruction buffer, storing the instructions in the PE internal instruction RAM and configuring the data RAM and function units under the control of the PE controller.

❸ *Execution.* PEs commence executing the application program, which involves loading/storing data from/to the data reconfigurable on-chip memory, dispatching data to downstream PE, and dynamically updating the number of remaining contexts and instructions in the instruction counter.

❹ *Workload Sensing.* When the number of remaining contexts and instructions reaches 0 or surpasses the busy threshold, the PE transitions to the *IDLE* or *BUSY* state and initiates workload sensing. It proactively sends a request for workload information to neighbor PEs through the task controller. Upon receiving the signal, the neighbor PEs transmit the remaining workload information to the *IDLE*/*BUSY* PE.

❺ *Tradeoff:* Following the reception of workload information from neighbor PEs, the scheduler within the *IDLE*/*BUSY* PE compares these information. It selects the PE with the maximum/minimum remaining workload as the target for task stealing/migration and tradeoffs based on the remaining workloads of both the *IDLE*/*BUSY* PE and the target PE. Subsequently, the scheduler determines the number of workloads to be stolen/migrated and sends the task stealing/migration request signal accordingly.

❻ *Stealing/Migration:* Once the target PE receives the task stealing/migration request signal, its task controller responds with an acknowledgment signal to the *IDLE*/*BUSY* PE. Concurrently, it manages the instruction RAM and data RAM to facilitate the transfer of instructions and related data with the *IDLE*/*BUSY* PE. Upon receiving the acknowledgment signal, the *IDLE*/*BUSY* PE coordinates the transfer of instructions and data with the target PE, ensuring seamless task stealing or migration. Note that both task stealing and migration operate at the granularity of a context. Furthermore, to ensure proper execution, task stealing/migration must target inactive contexts, which are contexts within a PE that have not yet begun execution.

❼ *Application Finish.* Upon completing the execution of all tasks assigned to it, each PE sets the finish register bit of its control register to 1. Simultaneously, the host core periodically polls the control registers of all PEs. Upon detecting that the finish register bits of all PEs have been set to 1, the host core terminates the execution of the application.

Synchronization. The dataflow-driven execution model in dataflow architectures naturally reduces load/store pressure and pollution potential in on-chip memory by enabling direct data transfer between PEs. Tasks are activated only when the necessary data from upstream PEs are available, implicitly ensuring write serialization and propagation across PEs. Initially, during task allocation, each PE receives its assigned tasks along with configuration information that records upstream and downstream data-dependent tasks and their associated PEs. This allows the PE to send activation signals and data to downstream PEs, as well as acknowledgment signals to upstream PEs upon receiving the required data. Additionally, in dynamic scheduling scenarios, including task stealing and migration, synchronization and data consistency are critical. These processes involve not only transferring relevant data, instructions, and configuration information from the source PE to the destination PE, but also updating the configuration information of upstream and downstream PEs to reflect the new PE identity of the stolen or migrated task. This ensures the correctness of the dataflow-driven execution and maintains synchronization across PEs.

4.3 Decentralized Dataflow-Driven Workload Balancing

Traditional static mapping faces challenges in achieving efficient workload balancing, which motivates our proposal for decentralized dynamic scheduling. PANDA adopts the *DFG balancing method* [19], which is a heuristic static mapping approach aimed at achieving load balancing through instruction mapping among DFG nodes. However, it is hard to generate an absolutely balanced DFG because: (1) the delay of each node during execution is unpredictable, particularly due to stalls caused by hazards or memory access; (2) allocating an equal number of instructions to each DFG node is costly and constrained by the applications itself, which often leads to non-convergence. To address these limitations, we propose decentralized dynamic scheduling to complement the static mapping. It's worth noting that the DFG balancing method can produce a relatively balanced load distribution across the PEs on the array. While potential workload imbalances may occur and are difficult to predict, the likelihood of neighboring PEs in a part of the array being in the same status (either *BUSY* or *IDLE*) is quite rare.

In a scenario where T_n tasks are mapped to an array of P_n PEs, workload balancing in the decentralized dataflow architecture entails two crucial steps: workload balancing trigger and workload balancing implementation.

Initially, PEs in the *imbalance* state trigger the workload balancing process. There are two types of *imbalance* states for PEs: *IDLE* (low-workload state) and *BUSY* (high-workload state). Accordingly, the PE in the *imbalance* state is formulated as follows:

$$\begin{aligned} \xi(imbalance) &= \xi(idle) \cup \xi(busy) \\ \xi(idle) &\cap \xi(busy) = \varnothing, \end{aligned} \tag{1}$$

where the PE in the *IDLE* state is formulated as follows:

$$\xi(idle) \neq \varnothing, \text{ if } C_n = 0 \text{ and } I_n = 0, \tag{2}$$

where the PE in the *BUSY* state is formulated as follows:

$$\xi(busy) \neq \varnothing, \text{ if } C_n > T_b, \tag{3}$$

where C_n represents the number of remaining contexts of the PE, I_n denotes the number of remaining instructions of the context being executed by the PE, T_b signifies the busy threshold, and $\xi(m)$, being non-empty, indicates that the PE is in the m state.

Additionally, the PE in the *EMPTY* state is formulated as follows:

$$\xi(empty) \neq \varnothing, \text{ if } C_n = 0 \text{ and } I_n \neq 0. \tag{4}$$

Whenever a PE transitions to the *IDLE* state, the busy threshold T_b is dynamically updated based on the number of *IDLE* and *EMPTY* PEs ($P_{n,i}$ and $P_{n,e}$) on the current array, as shown in Equation (5). The update depends on the number of *IDLE* and *EMPTY* PEs, as in both scenarios, the number of remaining contexts is 0. The task controller within each PE is responsible for storing and updating the busy threshold. Through this on-demand global threshold adjustment mechanism, we introduce a global hint to the decentralized scheduling, reducing the likelihood of prolonged idle states and mitigating the impact of local minima.

$$T_b = T_n \times \frac{P_{n,i} + P_{n,e}}{P_n} \tag{5}$$

Subsequently, workload balancing is achieved through task stealing for *IDLE* PEs and task migration for *BUSY* PEs.

The number of contexts obtained by the *IDLE* PE through task stealing, denoted as S_n, is given by Equation (6). The principle guiding task stealing for *IDLE* PEs is to alleviate the business of the neighbor PE with maximum remaining workload as much as possible within their capacity.

$$S_n = C_{n,t} - T_b \tag{6}$$

where $C_{n,t}$ represents the number of remaining contexts of the target PE for task stealing (the neighbor PE with maximum remaining workload).

The number of contexts lost by the *BUSY* PE through task migration, denoted as M_n, is given by Equation (7). The principle guiding task migration for *BUSY* PEs is to transition out of the *BUSY* state by migrating tasks to the neighbor PE with minimum remaining workload. Under this principle, if a situation arises where the remaining workload of a single PE is very high, the excessive workload will eventually be shared by the low-workload PEs on the array through continuous task migration. This process ensures more efficient utilization of resources across the array.

$$M_n = C_{n,b} - T_b \tag{7}$$

where $C_{n,b}$ represents the number of remaining contexts of the *BUSY* PE.

5 Evaluation Methodology

Setup. We implement all modules of PANDA using Verilog. We synthesize the design using Synopsys Design Compiler with a commercial 28 nm technology, meeting timing at a 1.25 GHz frequency. We use Synopsys PrimeTime PX for accurate power analysis. Table 3 shows the hardware design parameters, as well as the evaluated area and power breakdown of essential modules in PANDA. Evaluation shows that PANDA has an area footprint of 1.142 mm^2 in a 28 nm process, and consumes a maximum power of 1.217 W at 1.25 GHz.

Table 3. Design Parameters and Area and Power Breakdown (28nm)

Component (Parameter)		Area (mm^2)	Power (mW)
PE	**Function Unit (decoupled execution)**	0.016 (1.40%)	13.8 (1.13%)
	Controller Unit (decentral. control)	0.007 (0.61%)	12.9 (1.06%)
	Memory Unit (1KB inst/data RAM)	0.013 (1.14%)	16.4 (1.35%)
PE Array (4×4 decentralized PEs, SIMD)		0.574 (50.3%)	690.0 (56.7%)
Network on Chip (16 routers, 2D mesh)		0.098 (8.6%)	135.1 (11.1%)
Data Reconfig. On. Mem. (64KB total size)		0.349 (30.5%)	283.6 (23.3%)
Instruction Buffer (16KB total size)		0.121 (10.6%)	108.3 (8.9%)
PANDA (Total)		***1.142***	***1217***

Benchmarks. To evaluate our design, we select several real-world applications from REVEL [56], Plasticine [43], DFU [19], and MTDE [7]. For our experimentation, we deliberately opted for dual scales for each application. In the **small-scale (S)** scenario, the aggregate data size essential for application execution remains below the limits of the on-chip memory capacity. Conversely, the **large-scale (L)** case entails data size that surpasses the available on-chip memory capacity. We also assess the effectiveness of our method on graph applications where the number of edges equals (E) the number of vertices. In such graphs, each vertex typically has few neighbors. Additionally, we classify these applications as follows, based on their memory access and computation patterns. *(1) Regular Applications:* GEMM, STE, CONV, and FIR. These applications exhibit highly regular and fixed access and computation patterns. *(2) Near-Regular Applications:* SHA, ReLU, and SVD. These applications demonstrate relatively regular memory access and computation patterns but contain a small amount of discrete, small-scale parameter accesses and imperfect loops. *(3) Near-Irregular Applications:* FFT, LAG, and CHO. These applications feature complex hopping computations and include some data with a large span of memory access addresses. *(4) Irregular Applications:* BFS and PR. These applications involve imbalanced workloads, large amounts of random memory accesses across diverse memory regions, and redundant computations resulting from the traversal irregularity of edges and the update irregularity of vertex property [58]. Table 4 shows a list of applications and their characteristics.

Comparison Methodology. To quantify the performance of our design, we compare it to four state-of-the-art dataflow designs: REVEL [56], Plasticine [43], DFU [19], and MTDE [7]. REVEL comprises simple PEs in a systolic form and dataflow PEs capable of performing complex computations. It features hybrid static-dynamic task issuing and static task mapping. Plasticine decouples dedicated pattern compute units and memory units. It features static task issuing and static task mapping. DFU is a reconfigurable dataflow architecture for multi-batch data processing. It employs dynamic task issuing and static task mapping. MTDE is a multi-task dataflow engine that supports the elastic task scheduling scheme, featuring dynamic task issuing and dynamic task mapping. For fairness, these designs are extended to achieve similar peak performance and process. The clock frequencies are configured based on the specifications from their original articles.

6 Experimental Results

First, we evaluate the overall performance of PANDA architecture compared to the traditional dataflow architecture. Second, we evaluate the dataflow architecture of adaptive prefetching. In order to verify the optimization effect on the adaptive prefetching, for a fair comparison, we do

Table 4. Profile of Applications

Application	Domain	Data Sizes		Feature
		small	large	
GEMM	Scientific Computing	64×64	128×128	R
Stencil-3d		16×32×4	16×32×25K	R
SHA-256		8K Points	1M Points	NR
Conv-3d	Artificial Intelligence	8×8-32	32×32-32	R
ReLU-3d		8×8-32	32×32-32	NR
SVD		32×32	128×128	NR
FIR	Digital Signal Processing	8K Points	1G Points	R
Cholesky		32×32	128×128	NIR
FFT		1K Points	8K Points	NIR
Lagrange		128 Points	4K Points	NIR
BFS	Graph Processing	|V|=1K |E|=8K	|V|=9K |E|=5M,9K	IR
PageRank		|V|=1K |E|=8K	|V|=9K |E|=5M,9K	IR

In the table, all data types are 32-bit. R/NR/NIR/IR is short for regular, near-regular, near-irregular, and irregular.

not consider the decentralized scheduling. Subsequently, we evaluate the dataflow architecture of decentralized scheduling. Similarly, we do not consider the adaptive prefetching. In addition, we compare PANDA to four state-of-the-art dataflow designs: REVEL, Plasticine, DFU, and MTDE. Finally, we evaluate the overhead of the adaptive prefetching and decentralized scheduling.

6.1 Overall Performance

To evaluate the effectiveness of the methods we proposed, we implement the following four different configurations:

— *Base:* Our baseline is a dataflow architecture with full prefetching (SPM-only) and centralized scheduling.
— *Base + AP:* It only combines the baseline architecture with **A**daptive **P**refetching.
— *Base + DS:* It only combines the baseline architecture with **D**ecentralized **S**cheduling.
— *Base + AP + DS:* It combines the baseline architecture with **A**daptive **P**refetching and **D**ecentralized **S**cheduling.

Figure 9 demonstrates the effectiveness of our proposed methods in terms of performance improvements. **Adaptive prefetching (AP)** achieves an average performance improvement of 1.13× (small-scale) and 1.65× (large-scale) over the baseline, while decentralized scheduling (DS) achieves an average performance improvement of 1.40× (small-scale) and 1.67× (large-scale) over the baseline. The performance benefits of AP for workloads with irregular memory access patterns, such as BFS and PR, stem from the introduction of a cache suitable for handling the dynamic and unpredictable data access typical of such workloads. By combining the adaptive prefetching and decentralized scheduling, the performance of the dataflow architecture can be improved by 1.49× (small-scale) and 2.01× (large-scale). Moreover, for graph applications where the number of edges equals the number of vertices, the adaptive prefetching approach improves performance by 2.48× for BFS and 2.36× for PR, compared to full prefetching (SPM-only) method. Subsequently, we compare the performance gains obtained by each innovation point separately.

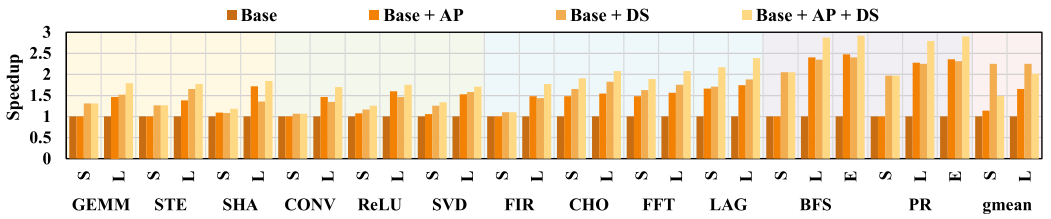

Fig. 9. Overall performance.

6.2 Advantages of Adaptive Prefetching

Our PANDA architecture adopts data reconfigurable on-chip memory to achieve adaptive data prefetching. To evaluate the advantages of adaptive prefetching, we replaced the data reconfigurable on-chip memory with DMA-assisted SPM. We also compared it to a cache-only configuration. Overall, we evaluate the following configurations:

— *SPM:* SPM with DMA support. SPM configuration: 64 KB total size, 16 banks, 128 lines, and 32B line size. DMA configuration: 400 ns initialization time.
— *Cache:* 64KB total size, 8-way set associativity, 128 sets, 64B line size, LRU replacement and write back.
— *PANDA MEM:* 64 KB total size, reconfigurable on-chip memory, SPM and cache logical organizations, SPM and cache shared SRAM.

Figure 10 shows the access latency for our 12 benchmarks with both large and small data sizes using DMA-assisted SPM, cache, and PANDA MEM, normalized to SPM. Compared to SPM with DMA, PANDA MEM effectively mitigates memory access latency, resulting in reductions of 18.2% (small-scale) and 55.1% (large-scale) on average. Additionally, PANDA MEM achieves an average of 47.0% (small-scale) and 16.7% (large-scale) latency improvement over the cache. Subsequently, we analyze these results in more detail.

For small-scale regular applications like small-scale GEMM, STE, CONV, and FIR, where all data are prefetchable, both PANDA MEM and SPM demonstrate equivalent low access latency. In contrast, the cache exhibits high access latency due to its lack of the bulk data prefetching mechanism. Similarly, in cases of small-scale SHA, ReLU, and SVD with a minor proportion of non-prefetchable data, PANDA MEM slightly outperforms SPM in optimizing access latency (by 12.1%, 9.9%, and 8.3%, respectively). Conversely, in scenarios where the proportion of non-prefetchable data is not negligible, PANDA MEM exhibits notable access latency reductions: 44.8% (small-scale FFT), 51.6% (large-scale FFT), 54.5% (small-scale LAG), 55.1% (large-scale LAG), 49.8% (small-scale CHO), 53.8% (large-scale CHO), and 55.1% (large-scale geomean). For irregular applications such as BFS and PR, where tasks are dynamically generated and activated during execution, SPM shows low access latency in small-scale scenarios but high access latency in large-scale scenarios, whereas cache and PANDA MEM show consistently low latency in both scenarios. The performance discrepancy of SPM in irregular applications primarily arises from its lack of the dynamic global addressing capability. In small-scale BFS and PR, where the data size fits within the on-chip memory and all data are prefetchable, SPM demonstrates low access latency. However, in large-scale BFS and PR, where the data size surpasses on-chip memory capacity and the task splitting is more inefficient compared to regular applications like GEMM, SPM shows high access latency. These results highlight the effectiveness of PANDA MEM in reducing access latency compared with SPM, particularly for applications with irregular access patterns. Compared to cache, PANDA MEM benefits from direct addressability to prefetchable data and the ability to efficiently prefetch bulk data. This gives

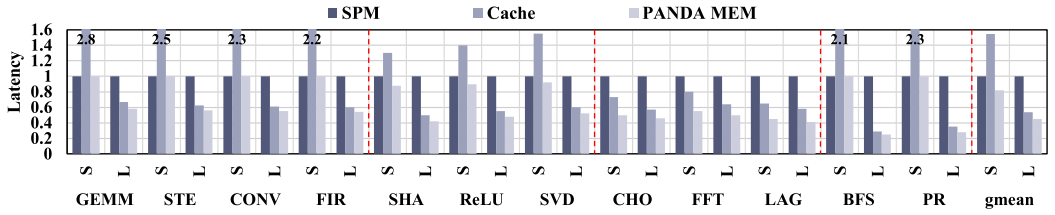

Fig. 10. Access latency comparison for SPM, cache, and PANDA MEM.

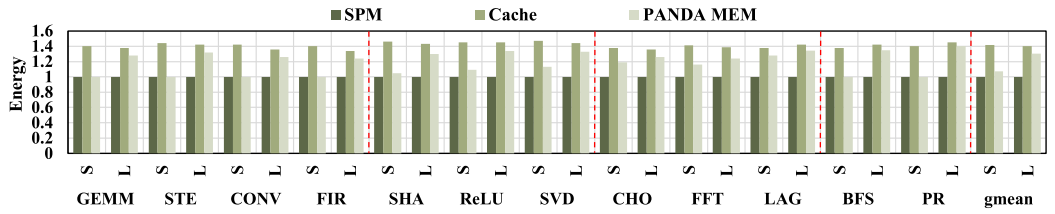

Fig. 11. Access energy comparison for SPM, cache, and PANDA MEM.

PANDA MEM a distinct edge in scenarios with substantial amounts of prefetchable data, leading to decreased access latency in such cases.

Figure 11 shows the access energy for our 12 benchmarks with both large and small data sizes using DMA-assisted SPM, cache, and PANDA MEM, normalized to SPM. We observe that the access energy of PANDA MEM exhibits nuanced behavior. When contrasted with SPM, PANDA MEM consumes more energy, with increases of 7.1% and 30.4% observed for small and large data sizes, respectively. In contrast, when compared to the cache, PANDA MEM exhibits energy advantages, achieving reductions of 32.2% and 7.7% for small and large data sizes, respectively. SPM has less hardware overhead than PANDA MEM and does not need to load non-prefetchable data from off-chip memory during execution. Compared to cache, PANDA MEM greatly reduces the need for tag lookups and minimizes conflict misses when accessing prefetchable data. Hence, PANDA MEM exhibits higher access energy than SPM but lower than cache.

6.3 Advantages of Decentralized Scheduling

Our PANDA architecture adopts decentralized dataflow PEs to achieve decentralized scheduling. To evaluate the advantages of decentralized scheduling, we replaced decentralized dataflow PEs with ordinary dataflow PEs and a centralized controller. Overall, we evaluate the following configurations:

— *Base PE:* 4×4 ordinary dataflow PEs under the control of a centralized controller, centralized scheduling.
— *PANDA PE:* 4×4 decentralized dataflow PEs, no centralized controller, decentralized scheduling.

Figure 12 shows the performance (in marker) and the computational resource utilization (in bar) comparison between the Base PE and PANDA PE, normalized to the centralized dataflow architecture. Compared to the Base PE, PANDA PE achieves an average performance improvement of 1.40× (small-scale) and 1.67× (large-scale). Moreover, PANDA PE achieves 1.23× (small-scale) and 1.34× (large-scale) utilization improvement over the Base PE. Further detailed analysis of these results follows.

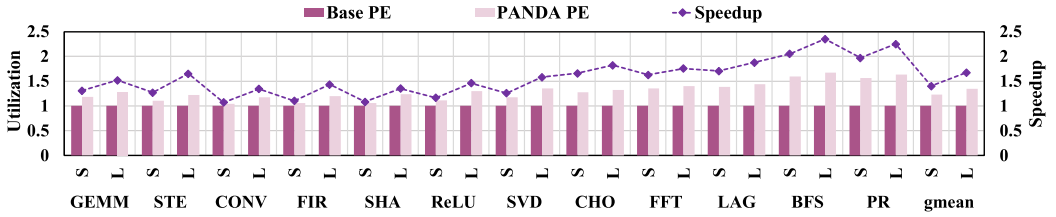

Fig. 12. Speedup (in Marker) and utilization (in Bar) comparisons for base PE and PANDA PE.

For regular and near-regular applications such as GEMM, STE, and SVD, which exhibit regular computational patterns and have limited optimization space for workload balancing in parallel processing, the relatively small performance gains primarily stem from the reduced communication and routing overhead between task iterations enabled by decentralized control. Additionally, as the data size and the number of task iterations increase, the performance improvement becomes more pronounced. For near-irregular applications such as FFT, LAG, and CHO, in addition to reduced communication overhead, performance gains are achieved through dynamic workload adjustment during execution. This adjustment allows for the combination of non-bottleneck tasks, overlapping the high latencies from frequent nested memory access and bottleneck tasks, thereby achieving higher performance. For irregular applications such as BFS and PR, where tasks are dynamically generated and activated during execution, PANDA PE not only maintains the above advantages compared to the Base PE but also enables more efficient dynamic workload adjustments during execution by improving hardware flexibility, resulting in better performance improvements. Similarly, the utilization improvement of PANDA PE compared to the Base PE is mainly attributed to reduced communication and routing overheads due to decentralized control and better workload balancing facilitated by increased hardware flexibility.

6.4 PANDA Outperforms State-of-the-Art Architectures

For comparison, we use four typical dataflow architectures: REVEL [56], Plasticine [43], DFU [19], and MTDE [7]. Table 5 shows the hardware comparisons of our architecture with other architectures.

Figure 13 illustrates the performance (in marker) comparison normalized to REVEL. Our design achieves an average performance improvement of 2.53× over REVEL, 1.90× over Plasticine, 1.38× over DFU, and 1.19× over MTDE. REVEL combines systolic PEs and dataflow PEs for hybrid-dataflow-driven scheduling, which allows better exploitation of application parallelism. However, it heavily relies on the compiler's heuristic search for the globally optimal timing decision due to the consideration of balancing between dataflow PEs and systolic PEs, limiting its performance gains. Plasticine employs non-dataflow-driven scheduling, focusing on a high-width SIMD design and decoupling compute and memory units to enhance performance by overlapping computation execution time and memory access latency. DFU enhances performance through full dataflow-driven scheduling and decouples computation into four pipeline stages, achieving higher levels of parallelism. These optimizations, however, are more effective for regular and near-regular applications, offering limited benefits for near-irregular and irregular applications due to the static scheduling employed by REVEL, Plasticine, and DFU, which lacks dynamic workload adjustment during execution. In contrast, MTDE and our proposed architecture, PANDA, employ dynamic scheduling, significantly improving performance, especially for near-irregular and irregular applications. PANDA stands out by adopting a decentralized design and utilizing highly flexible PEs for

Table 5. Hardware Comparison with State-of-the-art Architectures

Arch.	Scheduling Approach	On-chip Memory	Tech.	Area	Max. Power	Frequency	Peak Performance	Energy Efficiency
REVEL	Static + Hybrid	128KB (Shared SPM)	28 nm	2.709 mm^2	2.282 W	1.25 GHz	320 GFLOPS	25.99~137.29 GFLOPS/W
Plasticine	Static + Non-dataflow.	288KB (9 PMUs)	28 nm	3.760 mm^2	1.933 W	1 GHz	410 GFLOPS	59.25~150.86 GFLOPS/W
DFU	Static + Cen. Dataflow.	512KB(D)+128KB(C)	28 nm	10.28 mm^2	2.040 W	1.25 GHz	320 GFLOPS	88.06~154.25 GFLOPS/W
MTDE	Dynamic + Cen. Dataflow.	256KB (4-bank SPM)	28 nm	2.816 mm^2	2.104 W	0.8 GHz	410 GFLOPS	89.15~187.67 GFLOPS/W
PANDA	Dynamic + Dec. Dataflow.	64KB(D)+16KB(I)	28 nm	1.142 mm^2	1.217 W	1.25 GHz	320 GFLOPS	95.44~234.83 GFLOPS/W

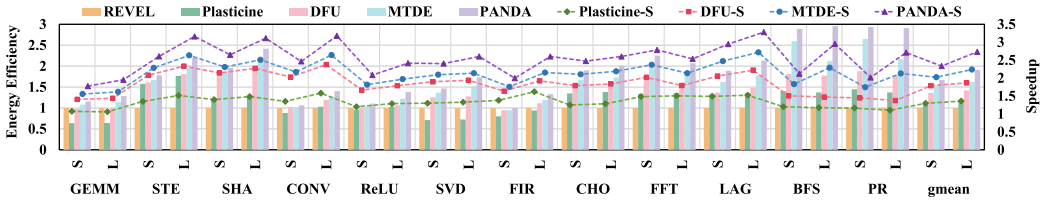

Fig. 13. Speedup (in marker) and energy efficiency (in Bar) comparisons with state-of-the-arts dataflow architectures.

more efficient dynamic scheduling. Additionally, it incorporates an adaptive prefetching approach that reduces memory access latency, further boosting performance compared to MTDE.

Figure 13 shows the energy efficiency (in bar) comparison normalized to REVEL. On average, our design achieves 1.79× energy efficiency improvement over REVEL, 1.60× over Plasticine, 1.28× over DFU, and 1.13× over MTDE. Task mapping significantly influences resource utilization. REVEL, Plasticine, and DFU, with static task mapping, generally exhibit lower utilization compared to MTDE and PANDA, which employ dynamic task mapping. Plasticine, with its high-width SIMD design, achieves high peak performance but demonstrates lower utilization when targeting small-scale applications, thus limiting energy efficiency in these scenarios. REVEL can fully exploit the hardware flexibility of its hybrid systolic-dataflow execution mechanism when applied to well-parallelized applications, achieving high utilization and energy efficiency in these scenarios. DFU, with its reconfigurable interconnections and decoupled execution mechanism, maintains relatively stable utilization and energy efficiency improvements across applications of different data sizes. However, due to the limitations of static mapping and rigid full prefetching, these architectures are less energy efficient for irregular and large-scale applications. MTDE employs dynamic mapping and achieves significant energy efficiency gains, particularly for irregular applications. PANDA also uses dynamic mapping and has a decentralized design. Compared to the centralized control of MTDE, decentralized control can significantly reduce the communication and routing overheads associated with dynamic scheduling and control, further improving energy efficiency.

6.5 Overhead

Finally, we evaluate the hardware overhead of the adaptive prefetching and decentralized scheduling. First is the adaptive prefetching. The hardware overhead of PANDA MEM is shown in Table 6, and PANDA MEM exhibits a total area overhead of 37.4% and 12.9% increase over SPM with DMA and cache of the same size, respectively. Next is the decentralized scheduling. The hardware overhead of PANDA PE is shown in Table 7, and PANDA PE exhibits a total area overhead of 6.5% increase over the Base PE of the same computing fabric.

7 Conclusion

In this article, we describe PANDA, a novel dataflow architecture with application-adaptive data prefetching and decentralized dataflow-driven dynamic scheduling. We implement PANDA in RTL

Table 6. Hardware Comparison of SPM, Cache, and PANDA MEM

	Component	Area (mm^2)	Power (mW)
SPM	**RAM Banks (64KB)**	0.212 (80.5%)	163.9 (81.1%)
	DMA & Control Logic	0.042 (16.5%)	39.6 (19.5%)
	Total	0.254	203.5
Cache	**Data Arrays (64KB)**	0.212 (68.6%)	163.9 (64.2%)
	Tag & Control Logic	0.097 (31.4%)	91.3 (35.8%)
	Total	0.309	255.2
PANDA MEM	**Shared RAM (64KB)**	0.212 (60.7%)	163.9 (57.8%)
	Control Logic	0.137 (39.3%)	119.7 (41.8%)
	Total	0.349	283.6

Table 7. Hardware Comparison of Base PE and PANDA PE

	Component	Area (mm^2)	Power (mW)
Base PE	**PEs (16 ordinary)**	0.478 (88.7%)	486.6 (74.7%)
	Controller	0.061 (11.3%)	165.1 (25.3%)
	Total	0.539	651.7
PANDA PE	**PEs (16 decentralized)**	0.574 (100%)	690.0 (100%)
	Controller (removed)	0 (0%)	0 (0%)
	Total	0.574	690.0

design and demonstrate that in a wide range of real-world applications, including scientific computing, artificial intelligence, digital signal processing, and graph processing algorithms, PANDA attains up to 2.53× performance and 1.79× energy efficiency improvement over the state-of-the-art dataflow architectures. Looking ahead, we will explore ways to efficiently support indirect memory access prefetching within our existing architecture. Our future work will focus on designing a low-cost hardware prefetcher to capture dynamic runtime information and optimizing the PANDA (software-aided) prefetching method through hardware-software co-design.

References

[1] Giovanni Ansaloni, Kazuyuki Tanimura, Laura Pozzi, and Nikil Dutt. 2012. Integrated kernel partitioning and scheduling for coarse-grained reconfigurable arrays. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 31, 12 (2012), 1803–1816. https://doi.org/10.1109/TCAD.2012.2209886

[2] ARM. 2023. Armv8-M Memory Model and Memory Protection User Guide. Retrieved August 7, 2024 from https://developer.arm.com/documentation/107565/0101/

[3] Bahar Asgari, Ramyad Hadidi, and Hyesoon Kim. 2020. ASCELLA: Accelerating sparse computation by enabling stream accesses to memory. In *Proceedings of the 23rd Conference on Design, Automation and Test in Europe (DATE'20)* (Grenoble, France). EDA Consortium, San Jose, CA, 318–321. https://doi.org/10.23919/DATE48585.2020.9116501

[4] Rajeshwari Banakar, Stefan Steinke, Bo-Sik Lee, Mahesh Balakrishnan, and Peter Marwedel. 2002. Scratchpad memory: A design alternative for cache on-chip memory in embedded systems. In *Proceedings of the 10th International Symposium on Hardware/Software Codesign (CODES 2002) (IEEE Cat. No.02TH8627)*. 73–78. https://doi.org/10.1145/774789.774805

[5] Brent Bohnenstiehl, Aaron Stillmaker, Jon J. Pimentel, Timothy Andreas, Bin Liu, Anh T. Tran, Emmanuel Adeagbo, and Bevan M. Baas. 2017. KiloCore: A 32-nm 1000-Processor computational array. *IEEE Journal of Solid-State Circuits* 52, 4 (2017), 891–902. https://doi.org/10.1109/JSSC.2016.2638459

[6] Doug Burger, James R. Goodman, and Alain Kägi. 1996. Memory bandwidth limitations of future microprocessors. In *Proceedings of the 23rd Annual International Symposium on Computer Architecture* (Philadelphia, PA) *(ISCA'96)*. ACM, New York, 78–89. https://doi.org/10.1145/232973.232983

[7] Longlong Chen, Jianfeng Zhu, Yangdong Deng, Zhaoshi Li, Jian Chen, Xiaowei Jiang, Shouyi Yin, Shaojun Wei, and Leibo Liu. 2021. An elastic task scheduling scheme on coarse-grained reconfigurable architectures. *IEEE Transactions on Parallel and Distributed Systems* 32, 12 (2021), 3066–3080. https://doi.org/10.1109/TPDS.2021.3084804

[8] Tianshi Chen, Zidong Du, Ninghui Sun, Jia Wang, Chengyong Wu, Yunji Chen, and Olivier Temam. 2014. DianNao: A small-footprint high-throughput accelerator for ubiquitous machine-learning. In *Proceedings of the 19th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'14)* (Salt Lake City, Utah, USA). ACM, New York, 269–284. https://doi.org/10.1145/2541940.2541967

[9] Tao Chen and G. Edward Suh. 2016. Efficient data supply for hardware accelerators with prefetching and access/execute decoupling. In *Proceedings of the 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO-49)* (Taipei, Taiwan). IEEE Press, Article 46, 12 pages. https://doi.org/10.1109/MICRO.2016.7783749

[10] Yu-Hsin Chen, Joel Emer, and Vivienne Sze. 2016. Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks. In *Proceedings of the 43rd International Symposium on Computer Architecture (ISCA'16)* (Seoul, Republic of Korea). IEEE Press, 367–379. https://doi.org/10.1109/ISCA.2016.40

[11] Hsiang-Yun Cheng, Chung-Hsiang Lin, Jian Li, and Chia-Lin Yang. 2010. Memory latency reduction via thread throttling. In *Proceedings of the 2010 43rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO'43)*. IEEE Computer Society, 53–64. https://doi.org/10.1109/MICRO.2010.39

[12] Vidushi Dadu and Tony Nowatzki. 2022. TaskStream: Accelerating task-parallel workloads by recovering program structure. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems* (Lausanne, Switzerland) *(ASPLOS'22)*. Association for Computing Machinery, New York, NY, USA, 1–13. https://doi.org/10.1145/3503222.3507706

[13] Vidushi Dadu, Jian Weng, Sihao Liu, and Tony Nowatzki. 2019. Towards general purpose acceleration by exploiting common data-dependence forms. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO'52)* (Columbus, OH). ACM, New York, 924–939. https://doi.org/10.1145/3352460.3358276

[14] Jinyi Deng, Xinru Tang, Jiahao Zhang, Yuxuan Li, Linyun Zhang, Boxiao Han, Hongjun He, Fengbin Tu, Leibo Liu, Shaojun Wei, Yang Hu, and Shouyi Yin. 2023. Towards efficient control flow handling in spatial architecture via architecting the control flow plane. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO'23)* (Toronto, ON, Canada). ACM, New York, 1395–1408. https://doi.org/10.1145/3613424.3614246

[15] Robert H. Dennard, Fritz H. Gaensslen, Hwa-Nien Yu, V. Leo Rideout, Ernest Bassous, and Andre R. LeBlanc. 1974. Design of ion-implanted MOSFET's with very small physical dimensions. *IEEE Journal of Solid-State Circuits* 9, 5 (1974), 256–268. https://doi.org/10.1109/JSSC.1974.1050511

[16] Jack B. Dennis. 1974. First version of a data flow procedure language. In *Programming Symposium*, B. Robinet (Ed.). Springer Berlin, Berlin, 362–376. https://doi.org/10.1007/3-540-06859-7_145

[17] Zhihua Fan, Wenming Li, Shengzhong Tang, Xuejun An, Xiaochun Ye, and Dongrui Fan. 2023. Improving utilization of dataflow architectures through software and hardware co-design. In *: Parallel Processing: Proceedings of the 29th International Conference on Parallel and Distributed Computing (Euro-Par 2023)* (Limassol, Cyprus, August 28 - September 1)). Springer-Verlag, Berlin, 245–259. https://doi.org/10.1007/978-3-031-39698-4_17

[18] Zhihua Fan, Wenming Li, Zhen Wang, Tianyu Liu, Haibin Wu, Yanhuan Liu, Meng Wu, Xinxin Wu, Xiaochun Ye, Dongrui Fan, et al.. 2023. Accelerating convolutional neural networks by exploiting the sparsity of output activation. *IEEE Transactions on Parallel and Distributed Systems* 34, 12 (2023), 3253–3265. https://doi.org/10.1109/TPDS.2023.3324934

[19] Zhihua Fan, Wenming Li, Zhen Wang, Yu Yang, Xiaochun Ye, Dongrui Fan, Ninghui Sun, and Xuejun An. 2024. Improving utilization of dataflow unit for multi-batch processing. *ACM Trans. Archit. Code Optim.* 21, 1, Article 17 (Feb 2024), 26 pages. https://doi.org/10.1145/3637906

[20] Mark Gebhart, Stephen W. Keckler, Brucek Khailany, Ronny Krashinsky, and William J. Dally. 2012. Unifying primary cache, scratch, and register file memories in a throughput processor. In *Proceedings of the 2012 45th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO-45)* (Vancouver, B.C., Canada). IEEE Computer Society, 96–106. https://doi.org/10.1109/MICRO.2012.18

[21] Tong Geng, Ang Li, Runbin Shi, Chunshu Wu, Tianqi Wang, Yanfei Li, Pouya Haghi, Antonino Tumeo, Shuai Che, Steve Reinhardt, and Martin C. Herbordt. 2020. AWB-GCN: A graph convolutional network accelerator with runtime workload rebalancing. In *Proceedings of the 2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO'20)*. 922–936. https://doi.org/10.1109/MICRO50266.2020.00079

[22] Graham Gobieski, Ahmet Oguz Atli, Kenneth Mai, Brandon Lucia, and Nathan Beckmann. 2021. Snafu: An ultra-low-power, energy-minimal CGRA-generation framework and architecture. In *Proceedings of the 48th Annual International Symposium on Computer Architecture* (Virtual Event, Spain) *(ISCA'21)*. IEEE Press, 1027–1040. https://doi.org/10.1109/ISCA52012.2021.00084

[23] Graham Gobieski, Amolak Nagi, Nathan Serafin, Mehmet Meric Isgenc, Nathan Beckmann, and Brandon Lucia. 2019. MANIC: A vector-dataflow architecture for ultra-low-power embedded systems. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO'52)* (Columbus, OH, USA). ACM, New York, 670–684. https://doi.org/10.1145/3352460.3358277

[24] Graham Gobieski, Amolak Nagi, Nathan Serafin, Mehmet Meric Isgenc, Nathan Beckmann, and Brandon Lucia. 2019. MANIC: A vector-dataflow architecture for ultra-low-power embedded systems. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture* (Columbus, OH,) *(MICRO'52)*. ACM, New York, 670–684. https://doi.org/10.1145/3352460.3358277

[25] Tae Jun Ham, Juan L. Aragón, and Margaret Martonosi. 2015. DeSC: Decoupled supply-compute communication management for heterogeneous architectures. In *Proceedings of the 48th International Symposium on Microarchitecture (MICRO-48)* (Waikiki, Hawaii). ACM' New York, 191–203. https://doi.org/10.1145/2830772.2830800

[26] Tae Jun Ham, Lisa Wu, Narayanan Sundaram, Nadathur Satish, and Margaret Martonosi. 2016. Graphicionado: A high-performance and energy-efficient accelerator for graph analytics. In *Proceedings of the 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO-49)* (Taipei, Taiwan). IEEE Press, Article 56, 13 pages. https://doi.org/10.1109/MICRO.2016.7783759

[27] Sangyeol Kang and Alexander G. Dean. 2012. Leveraging both data cache and scratchpad memory through synergetic data allocation. In *Proceedings of the 2012 IEEE 18th Real Time and Embedded Technology and Applications Symposium*. 119–128. https://doi.org/10.1109/RTAS.2012.22

[28] Manupa Karunaratne, Aditi Kulkarni Mohite, Tulika Mitra, and Li-Shiuan Peh. 2017. HyCUBE: A CGRA with reconfigurable single-cycle multi-hop interconnect. In *Proceedings of the 54th Annual Design Automation Conference 2017 (DAC'17)* (Austin, TX). ACM, New York, Article 45, 6 pages. https://doi.org/10.1145/3061639.3062262

[29] Manupa Karunaratne, Dhananjaya Wijerathne, Tulika Mitra, and Li-Shiuan Peh. 2019. 4D-CGRA: Introducing branch dimension to spatio-temporal application mapping on CGRAs. In *Proceedings of the 2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD'19)*. 1–8. https://doi.org/10.1109/ICCAD45719.2019.8942148

[30] Changkyu Kim, Simha Sethumadhavan, M. S. Govindan, Nitya Ranganathan, Divya Gulati, Doug Burger, and Stephen W. Keckler. 2007. Composable lightweight processors. In *Proceedings of the 40th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO 40)*. IEEE Computer Society, 381–394. https://doi.org/10.1109/MICRO.2007.41

[31] Rakesh Komuravelli, Matthew D. Sinclair, Johnathan Alsop, Muhammad Huzaifa, Maria Kotsifakou, Prakalp Srivastava, Sarita V. Adve, and Vikram S. Adve. 2015. Stash: Have your scratchpad and cache it too. In *Proceedings of the 42nd Annual International Symposium on Computer Architecture (ISCA'15)* (Portland, Oregon). ACM, New York, 707–719. https://doi.org/10.1145/2749469.2750374

[32] Chris Lattner and Vikram Adve. 2004. LLVM: A compilation framework for lifelong program analysis & transformation. In *Proceedings of the International Symposium on Code Generation and Optimization: Feedback-Directed and Runtime Optimization (CGO'04)* (Palo Alto, CA). IEEE Computer Society, 75. https://doi.org/10.1109/CGO.2004.1281665

[33] Tianyu Liu, Wenming Li, and Zhihua Fan. 2023. DFGC: DFG-aware NoC control based on time stamp prediction for dataflow architecture. In *Proceedings of the 2023 IEEE 41st International Conference on Computer Design (ICCD'23)*. 432–439. https://doi.org/10.1109/ICCD58817.2023.00071

[34] Mahim Mishra, Timothy J. Callahan, Tiberiu Chelcea, Girish Venkataramani, Seth C. Goldstein, and Mihai Budiu. 2006. Tartan: Evaluating spatial computation for whole program execution. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS XII)* (San Jose, CA). ACM, New York, 163–174. https://doi.org/10.1145/1168857.1168878

[35] Gregory E. Moore. 1998. Cramming more components onto integrated circuits. *Proc. IEEE* 86, 1 (1998), 82–85. https://doi.org/10.1109/JPROC.1998.658762

[36] Quan M. Nguyen and Daniel Sanchez. 2021. Fifer: Practical acceleration of irregular applications on reconfigurable architectures. In *Proceedings of the MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO'21)* (Virtual Event, Greece). ACM, New York, 1064–1077. https://doi.org/10.1145/3466752.3480048

[37] Tony Nowatzki, Vinay Gangadhar, Newsha Ardalani, and Karthikeyan Sankaralingam. 2017. Stream-dataflow acceleration. In *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA'17)* (Toronto, ON, Canada). ACM, New York, 416–429. https://doi.org/10.1145/3079856.3080255

[38] NVIDIA. 2023. NVIDIA H100 Tensor Core Hopper Whitepaper. Retrieved August 7, 2024 from https://resources.nvidia.com/en-us-tensor-core/

[39] Angshuman Parashar, Michael Pellauer, Michael Adler, Bushra Ahsan, Neal Crago, Daniel Lustig, Vladimir Pavlov, Antonia Zhai, Mohit Gambhir, Aamer Jaleel, et al. 2013. Triggered instructions: A control paradigm for spatially-programmed architectures. In *Proceedings of the 40th Annual International Symposium on Computer Architecture (ISCA'13)* (Tel-Aviv, Israel). ACM, New York, 142–153. https://doi.org/10.1145/2485922.2485935

[40] Hyunchul Park, Yongjun Park, and Scott Mahlke. 2009. Polymorphic pipeline array: A flexible multicore accelerator with virtualized execution for mobile multimedia applications. In *Proceedings of the 42nd Annual IEEE/ACM*

International Symposium on Microarchitecture (MICRO 42) (New York, New York). ACM, New York, 370–380. https://doi.org/10.1145/1669112.1669160

[41] Julian Pavon, Ivan Vargas Valdivieso, Adrian Barredo, Joan Marimon, Miquel Moreto, Francesc Moll, Osman Unsal, Mateo Valero, and Adrian Cristal. 2021. VIA: A smart scratchpad for vector units with application to sparse matrix computations. In *Proceedings of the 2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA'21)*. 921–934. https://doi.org/10.1109/HPCA51647.2021.00081

[42] Michael Pellauer, Yakun Sophia Shao, Jason Clemons, Neal Crago, Kartik Hegde, Rangharajan Venkatesan, Stephen W. Keckler, Christopher W. Fletcher, and Joel Emer. 2019. Buffets: An efficient and composable storage idiom for explicit decoupled data orchestration. In *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'19)* (Providence, RI). ACM, New York, 137–151. https://doi.org/10.1145/3297858.3304025

[43] Raghu Prabhakar, Yaqi Zhang, David Koeplinger, Matt Feldman, Tian Zhao, Stefan Hadjis, Ardavan Pedram, Christos Kozyrakis, and Kunle Olukotun. 2017. Plasticine: A reconfigurable architecture for parallel paterns. In *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA'17)* (Toronto, ON, Canada). ACM, New York, 389–402. https://doi.org/10.1145/3079856.3080256

[44] Shantian Qin, Wenming Li, Zhihua Fan, Zhen Wang, Tianyu Liu, Haibin Wu, Kunming Zhang, Xuejun An, Xiaochun Ye, and Dongrui Fan. 2023. ROMA: A reconfigurable on-chip memory architecture for multi-core accelerators. In *Proceedings of the 2023 IEEE International Conference on High Performance Computing (HPCC'23)*. 49–57. https://doi.org/10.1109/HPCC-DSS-SmartCity-DependSys60770.2023.00017

[45] Nauman Rafique, Won-Taek Lim, and Mithuna Thottethodi. 2007. Effective management of DRAM bandwidth in multicore processors. In *Proceedings of the 16th International Conference on Parallel Architecture and Compilation Techniques (PACT'07)*. 245–258. https://doi.org/10.1109/PACT.2007.4336216

[46] Ali Sedaghati, Milad Hakimi, Reza Hojabr, and Arrvindh Shriraman. 2022. X-cache: A modular architecture for domain-specific caches. In *Proceedings of the 49th Annual International Symposium on Computer Architecture* (New York, New York) *(ISCA'22)*. ACM, New York, 396–409. https://doi.org/10.1145/3470496.3527380

[47] Steven Swanson, Andrew Schwerin, Martha Mercaldi, Andrew Petersen, Andrew Putnam, Ken Michelson, Mark Oskin, and Susan J. Eggers. 2007. The WaveScalar architecture. *ACM Trans. Comput. Syst.* 25, 2, Article 4 (May 2007), 54 pages. https://doi.org/10.1145/1233307.1233308

[48] Cheng Tan, Nicolas Bohm Agostini, Tong Geng, Chenhao Xie, Jiajia Li, Ang Li, Kevin J. Barker, and Antonino Tumeo. 2022. DRIPS: Dynamic rebalancing of pipelined streaming applications on CGRAs. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA'22)*. 304–316. https://doi.org/10.1109/HPCA53966.2022.00030

[49] Cheng Tan, Chenhao Xie, Tong Geng, Andres Marquez, Antonino Tumeo, Kevin Barker, and Ang Li. 2021. ARENA: Asynchronous reconfigurable accelerator ring to enable data-centric parallel computing. *IEEE Transactions on Parallel and Distributed Systems* 32, 12 (2021), 2880–2892. https://doi.org/10.1109/TPDS.2021.3081074

[50] Po-An Tsai, Nathan Beckmann, and Daniel Sanchez. 2017. Jenga: Software-defined cache hierarchies. In *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA'17)* (Toronto, ON, Canada). ACM, New York, 652–665. https://doi.org/10.1145/3079856.3080214

[51] Po-An Tsai, Yee Ling Gan, and Daniel Sanchez. 2018. Rethinking the memory hierarchy for modern languages. In *Proceedings of the 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO-51)* (Fukuoka, Japan). IEEE Press, 203–216. https://doi.org/10.1109/MICRO.2018.00025

[52] Zhengrong Wang, Christopher Liu, Aman Arora, Lizy John, and Tony Nowatzki. 2023. Infinity stream: Portable and programmer-friendly in-/near-memory fusion. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (Vancouver, BC, Canada) *(ASPLOS'23)*. Association for Computing Machinery, New York, NY, USA, 359–375. https://doi.org/10.1145/3582016.3582032

[53] Zhengrong Wang and Tony Nowatzki. 2019. Stream-based memory access specialization for general purpose processors. In *Proceedings of the 46th International Symposium on Computer Architecture (ISCA'19)* (Phoenix, AZ) . ACM, New York, 736–749. https://doi.org/10.1145/3307650.3322229

[54] Zhengrong Wang, Jian Weng, Sihao Liu, and Tony Nowatzki. 2022. Near-stream computing: General and transparent near-cache acceleration. In *Proceedings of the 2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA'22)*. 331–345. https://doi.org/10.1109/HPCA53966.2022.00032

[55] Jian Weng, Sihao Liu, Vidushi Dadu, Zhengrong Wang, Preyas Shah, and Tony Nowatzki. 2020. DSAGEN: Synthesizing programmable spatial accelerators. In *Proceedings of the ACM/IEEE 47th Annual International Symposium on Computer Architecture* (Virtual Event) *(ISCA'20)*. IEEE Press, 268–281. https://doi.org/10.1109/ISCA45697.2020.00032

[56] Jian Weng, Sihao Liu, Zhengrong Wang, Vidushi Dadu, and Tony Nowatzki. 2020. A hybrid systolic-dataflow architecture for inductive matrix algorithms. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA'20)*. 703–716. https://doi.org/10.1109/HPCA47549.2020.00063

[57] Wm. A. Wulf and Sally A. McKee. 1995. Hitting the memory wall: implications of the obvious. *SIGARCH Comput. Archit. News* 23, 1 (Mar 1995), 20–24. https://doi.org/10.1145/216585.216588

[58] Mingyu Yan, Xing Hu, Shuangchen Li, Abanti Basak, Han Li, Xin Ma, Itir Akgun, Yujing Feng, Peng Gu, Lei Deng, et al. 2019. Alleviating irregularity in graph analytics acceleration: A hardware/software co-design approach. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO'52)* (Columbus, OH, USA). ACM, New York, 615–628. https://doi.org/10.1145/3352460.3358318

[59] Guowei Zhang, Nithya Attaluri, Joel S. Emer, and Daniel Sanchez. 2021. Gamma: Leveraging Gustavson's algorithm to accelerate sparse matrix multiplication. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'21)* (Virtual, USA). ACM, New York, 687–701. https://doi.org/10.1145/3445814.3446702

[60] Shixuan Zheng, Xianjue Zhang, Leibo Liu, Shaojun Wei, and Shouyi Yin. 2022. Atomic dataflow based graph-level workload orchestration for scalable DNN accelerators. In *Proceedings of the 2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA'22)*. 475–489. https://doi.org/10.1109/HPCA53966.2022.00042

Received 8 August 2024; revised 22 January 2025; accepted 24 February 2025